SlotCrossAttention-CEM

Hyperparameter Sweep for MIA Defense in Split Learning on FaceScrub

Group Meeting Presentation

March 2026

Overview: What Are We Doing?

Goal

Beat the original paper's SOTA defense against Model Inversion Attacks (MIA) on FaceScrub (530 identities). **Baseline Inference SSIM $\approx$ 0.532** (lower = better privacy).

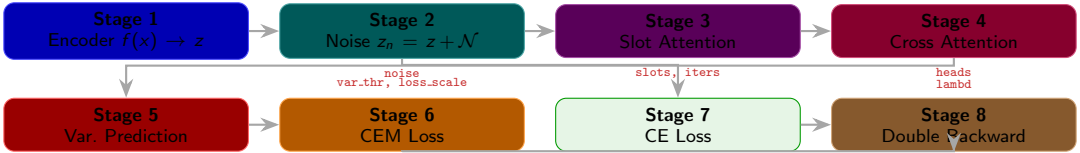
Our Innovation: SCA_new

- Replace hard KMeans clustering with **differentiable Slot Attention**
- End-to-end trainable CEM defense
- No per-epoch full-dataset re-clustering

Experiment: `run3.sh`

- **20 experiments**, 6 groups (A–F)
- Systematically sweep key hyperparameters
- Each experiment: 300–360 epochs training + 50 epochs attack evaluation

Architecture Pipeline: 8 Stages



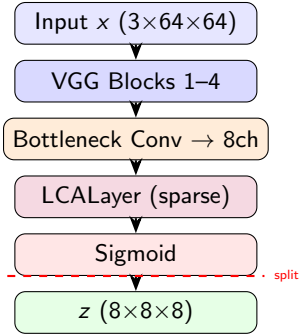
Cross-stage controls: `warmup` → gates Stages 3–6 (skipped during warmup) `epochs` → total training length

Stage 1: Encoder $f(x) \to z$ [Fixed across all experiments]

Architecture: VGG11-BN-SGM

- VGG11 backbone with BatchNorm
- Split at `cutlayer=4`
- Bottleneck: Conv $\to$ 8 channels (`noRELU_C8S1`)
- **LCALayer**: Locally Competitive Algorithm for sparse coding
- **Sigmoid** activation (not ReLU)

No parameters changed here.
All 20 experiments use the identical encoder architecture.



Stage 2: Gaussian Noise Injection [noise]

Mechanism

$$z_n = z + \mathcal{N}(0, \sigma^2), \quad \sigma = \texttt{noise}$$

Applied during **both training and inference**. Protects the transmission channel.

Effect of increasing noise:

- ↑ Attacker's decoder must "de-noise" — harder reconstruction
- ↑ CEM threshold auto-increases ($\text{thr} = \text{vt} \times \sigma^2$)
- ↓ Classifier receives noisier signal — accuracy may drop

Settings across experiments:

Config	noise (σ)	Relative
Baseline	0.05	1.0×
exp01–03	0.05	1.0×
exp04–05	**0.08**	1.6×
exp06,15,19,20	**0.06**	1.2×
exp07	**0.10**	2.0×
exp12,14,16,18	**0.07**	1.4×
exp17	**0.08**	1.6×

Key insight: noise has a **dual role** — it protects transmission (Stage 2) AND raises the CEM threshold (Stage 6).

Stage 3: Slot Attention (Soft Clustering) [slots, iters]

Algorithm (differentiable, replaces KMeans):

1. Initialize S slot prototypes from learnable μ, σ
2. **Repeat** $K =$ iters times:
   - $\mathbf{q} = W_q(\text{slots}), \quad \mathbf{k} = W_k(z), \quad \mathbf{v} = W_v(z)$
   - $\text{attn} = \text{softmax}(\mathbf{q} \cdot \mathbf{k}^\top / \sqrt{d})$
   - Normalize: competitive assignment
   - $\text{updates} = \text{attn} \cdot \mathbf{v}$
   - $\text{slots} = \text{GRU}(\text{updates}, \text{slots}_{prev})$
   - $\text{slots} \mathrel{+}= \text{MLP}(\text{slots})$
3. Output: S refined slot vectors

slots (Group C: exp08–11):

Exp	slots	Rationale
exp09	**4**	Lower bound test
baseline	8	Default
exp18	**10**	Moderate increase
exp11,15,20	**12**	Finer clustering
exp08	**16**	Upper bound test

iters (GRU iterations):

Exp	iters	Rationale
baseline	3	Default
exp11,15,16,18,20	**4**	+1 refinement
exp10,19	**5**	Max refinement

Stage 4: Cross Attention Refinement [heads]

Mechanism:

- Query = slot vectors from Stage 3
- Key/Value = original features z
- Multi-head attention with heads independent heads
- **Tanh gating**: $y = \text{slots} + \tanh(\alpha) \cdot \text{CrossAttn}(\text{slots}, z)$
- Followed by gated FFN: $y \mathrel{+}= \tanh(\beta) \cdot \text{FFN}(y)$

Constraint: heads must divide feature_dim=8. Valid: 1,2,4,8.

How heads affects computation:

heads	head_dim	Perspective
2	4	Fewer but wider
4	2	Default balance
8	1	Many but narrow

Only exp18 changes heads:

- heads = 2 (down from 4)
- Each head sees 4-dim subspace
- Tests: does a wider view per head help slot refinement?

Tanh gating: α, β are learnable scalars (init 0.1) — prevents cross-attention from dominating early; the model gradually "opens the gate" as it learns.

Stage 5: Variance Prediction [No tunable hyperparameters]

Mechanism

$$\mu_s = \texttt{head_mu}(\text{slots}) = W_\mu \cdot \text{slots} + b_\mu, \qquad \text{var}_s = \text{Softplus}(W_v \cdot \text{slots} + b_v) + 10^{-4}$$

Key design choices:

- Variance is **predicted by a neural network**, not computed geometrically
- **Softplus** ensures positivity
- Running statistics (EMA, β=0.99) for training stability

Contrast with original CEM:

Original (KMeans)	**Ours (SCA)**
$\text{var} = \frac{1}{N}\sum(z_i - c_k)^2$	$\text{var} = \text{Softplus}(W \cdot \text{slots})$
Geometric distance	Learned representation
Non-differentiable	Fully differentiable
1-epoch stale	Real-time

No hyperparameters tuned here — prediction heads are fixed architecture. Quality depends on upstream Stages 3–4 (`slots`, `iters`, `heads`).

Stage 6: CEM Loss Computation [var_threshold, loss_scale]

Loss Formula

$$\mathcal{L}_{\text{CEM}} = \frac{1}{|\mathcal{C}|}\sum_{c\in\mathcal{C}}\frac{1}{|c|}\sum_{i\in c}\text{ReLU}\Big(\underbrace{\log(\text{thr})}_{\text{target}} - \underbrace{\log(\bar{v}_i+\epsilon)}_{\text{predicted var}}\Big), \quad \text{thr} = \texttt{var_thr} \times \texttt{noise}^2 + \epsilon$$

var_threshold settings:

Exp	vt	noise	Threshold
Baseline	0.15	0.05	3.75e-4
exp01–03	0.15	0.05	3.75e-4
exp04–05	**0.20**	0.08	**1.28e-3**
exp06	**0.25**	0.06	**9.00e-4**
exp07	**0.20**	0.10	**2.00e-3**
exp16,17,20	**0.25**	var	**high**

loss_scale (pre-backward scaling):

$$\mathcal{L}_{\text{CEM}}^{\text{scaled}} = \texttt{loss_scale} \times \mathcal{L}_{\text{CEM}}$$

Config	ls	Effect
Baseline	0.5	Halved!
Most exp	**1.0**	Full strength
exp03,05,07,16,17	**0.8**	Slight control
exp14,20	**1.2**	Amplified

Baseline problem: loss_scale=0.5 was halving CEM before it even reached the double-backward. All experiments fix this to ≥ 0.8.

Stages 7 & 8: Classification + Double Backward [lambd]

Stage 7 (fixed):

$$z_n \xrightarrow{f_{\text{tail}}} \hat{y}, \quad \mathcal{L}_{\text{CE}} = \text{CE}(\hat{y}, y)$$

Standard cross-entropy. No parameters changed.

Stage 8: Double Backward

1. $\mathcal{L}_{\text{CEM}}$.backward(retain_graph) — save $g_{\text{cem}} = \nabla_{\theta_f} \mathcal{L}_{\text{CEM}}$
2. Save attention module gradients
3. `optimizer.zero_grad()`
4. $\mathcal{L}_{\text{CE}}$.backward()
5. Manual addition: ∇_{θ_f} += `lambd` $\times\, g_{\text{cem}} \times s(lr)$

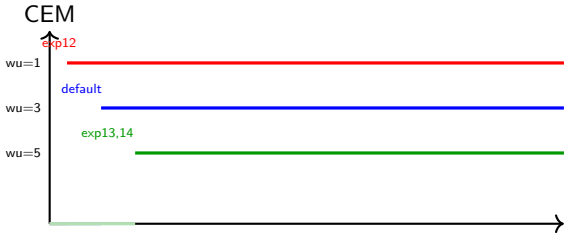
$$s(lr) = \begin{cases} 0.001/lr & lr > 0.00041 \text{ (suppressed early)} \\ 1.0 & lr \leq 0.00041 \text{ (full power late)} \end{cases}$$

Effective CEM Strength = `lambd` × `loss_scale` (post-warmup, full power)

	Baseline	exp01	exp02	exp03	exp15	exp17	exp19	**exp20**
Effective	8.0	24.0	32.0	38.4	40.0	38.4	40.0	**43.2**

Cross-Stage Control: warmup and epochs

warmup: When does CEM activate?



Stages 3–6 are **completely skipped** during warmup.
⇒ Pure classification for first warmup epochs.

epochs: CosineAnnealing & CEM full-power

Adaptive scaling: CEM is suppressed when $lr > 0.00041$ (early training).
CEM reaches full power only in the final ∼5% of epochs:

epochs	Full-power	Extra
240 (baseline)	∼12 ep	—
300	∼15 ep	+25%
360 (exp19)	∼18 ep	+50%

exp19 uses 360 epochs specifically to extend the CEM full-power training window.

Experiment Groups: Design Rationale

Group	Experiments	Research Question	Stages Tested
A	exp01–03	Does increasing `lambd` & `loss_scale` alone improve defense?	6, 8
B	exp04–07	Can higher `noise` + stricter `var_threshold` synergize?	2, 6
C	exp08–11	How do `slots` and `iters` affect clustering quality?	3
D	exp12–14	When should CEM activate? (`warmup` sensitivity)	3–6 gate
E	exp15–18	Do multi-dimensional improvements compound?	2, 3, 4, 6, 8
F	exp19–20	Best candidates to beat SOTA	All

Strategy: Groups A–D isolate individual factors via **controlled experiments**.
Groups E–F combine the best configurations from A–D to find optimal combinations.

Group A (exp01–03): CEM Gradient Strength Sweep

Question: Baseline effective strength = 8.0 is too weak. What happens if we push to 24–38?

	lambd	ls	**Eff. Str.**	noise	vt	slots	iters
Baseline	16	0.5	8.0	0.05	0.15	8	3
exp01	**24**	**1.0**	24.0	0.05	0.15	8	3
exp02	**32**	**1.0**	32.0	0.05	0.15	8	3
exp03	**48**	**0.8**	38.4	0.05	0.15	8	3

- **Only Stages 6 & 8 are changed**. All other stages identical to baseline.
- exp03 uses `ls`=0.8 (not 1.0) because $48 \times 1.0 = 48$ might be too aggressive and crash classification accuracy.
- **Expected**: monotonic SSIM decrease (better privacy) with possible accuracy trade-off at exp03.

Group B (exp04–07): Noise + Threshold Push

Question: Can we improve defense by pushing `noise` (Stage 2) and `var_threshold` (Stage 6) simultaneously?

	`lambd`	`noise`	`vt`	`ls`	**Eff. Str.**	**Threshold**
Baseline	16	0.05	0.15	0.5	8.0	3.75e-4
exp04	24	**0.08**	**0.20**	1.0	24.0	**1.28e-3**
exp05	32	**0.08**	**0.20**	0.8	25.6	**1.28e-3**
exp06	24	**0.06**	**0.25**	1.0	24.0	**9.00e-4**
exp07	32	**0.10**	**0.20**	0.8	25.6	**2.00e-3**

- **exp06**: minimal noise increase (0.05→0.06) but maximum `vt` (0.25) — tests "strict threshold, low noise" strategy
- **exp07**: extreme noise (0.10 = 2× baseline) — tests privacy at the cost of classification accuracy
- Threshold increases 2.4× to 5.3× vs baseline

Group C (exp08–11): Slot Attention Architecture Search

Question: How do `slots` and `iters` affect clustering quality? (Controlled experiment: only Stage 3 changes)

	`slots`	`iters`	`heads`	**Eff. Str.**	**What Changed**
Baseline	8	3	4	8.0	—
exp08	**16**	3	4	24.0	2× prototypes
exp09	**4**	3	4	24.0	0.5× prototypes
exp10	8	**5**	4	32.0	+2 GRU iterations
exp11	**12**	**4**	4	24.0	Balanced middle

Slots trade-off:

- Too few (4): coarse clustering, 530 classes mapped to 4 prototypes
- Too many (16): slot collapse risk (some slots "die" and never get assigned features)

Iterations trade-off:

- 3 iterations: may not fully converge slot competition
- 5 iterations: more refined, but +67% compute and potential vanishing gradients

Group D (exp12–14): Warmup Sensitivity

Question: How much does the CEM activation timing matter?

	wu	lambd	noise	vt	ls	**Strategy**
Baseline	3	16	0.05	0.15	0.5	Default
exp12	**1**	24	0.07	0.20	1.0	Aggressive: CEM from epoch 2
exp13	**5**	32	0.06	0.20	1.0	Conservative: classifier first
exp14	**5**	24	0.07	0.20	**1.2**	Late start, compensate with ↑ls

warmup=1 (exp12):

- CEM starts when accuracy $< 5\%$
- Risk: encoder learns to scatter features before learning to classify
- Possible benefit: features never "crystallize" into attack-friendly patterns

warmup=5 + ls=1.2 (exp14):

- Classifier converges for 5 epochs first
- CEM "late arrival" compensated by 20% amplified loss scale
- Effective strength: $24 \times 1.2 = 28.8$

Groups E & F: Combined Configs and Best Candidates

E (exp15–18): Multi-dimensional improvements **F (exp19–20)**: Final candidates

	λ	σ	vt	ls	sl	hd	it	wu	ep	**Eff.**
Baseline	16	.05	.15	0.5	8	4	3	3	240	8.0
exp15	**40**	.06	.20	1.0	**12**	4	**4**	3	300	40.0
exp16	36	.07	**.25**	0.8	8	4	**4**	**5**	300	28.8
exp17	**48**	**.08**	**.25**	0.8	8	4	3	**5**	300	38.4
exp18	28	.07	.20	1.0	**10**	**2**	**4**	3	300	28.0
exp19	**40**	.06	.20	1.0	8	4	**5**	3	**360**	40.0
exp20	36	.06	**.25**	**1.2**	**12**	4	**4**	**5**	300	**43.2**

- **exp19**: 360 epochs $\Rightarrow$ 18 epochs full-power CEM (vs 12 baseline)
- **exp20**: highest eff. strength (43.2 = 5.4$\times$ baseline), strictest threshold, most slots

Summary: Parameter $\leftrightarrow$ Stage Mapping

Parameter	Stage	What It Controls	$\uparrow$ Benefit	$\uparrow$ Risk
`noise` (σ)	2, 6	Noise std + CEM threshold	Harder reconstruction	Lower accuracy
`slots`	3	# soft-cluster prototypes	Finer clustering	Slot collapse
`iters`	3	GRU refinement rounds	Better convergence	Computation cost
`heads`	4	Multi-head perspectives	More diverse views	Narrow head dims
`var_thr`	6	Variance strictness	Stronger dispersion	Training instability
`loss_scale`	6$\rightarrow$8	CEM loss amplifier	Stronger CEM signal	Accuracy loss
`lambd`	8	Gradient injection coeff.	Stronger encoder push	CE grad drowned
`warmup`	3–6	CEM activation delay	Stable classification	Less defense time
`epochs`	all	Training duration	Longer full-power CEM	Wall-clock time

Success Criterion

Find a configuration where **Inference SSIM** $<$ **0.532** while maintaining reasonable classification accuracy ($>$60%). Results pending.

Thank You

Experiments are currently running on the server.
Results will be available in `run3log_*/summary.csv`

20 experiments $\times$ $\sim$5h each $\approx$ 100 hours total